

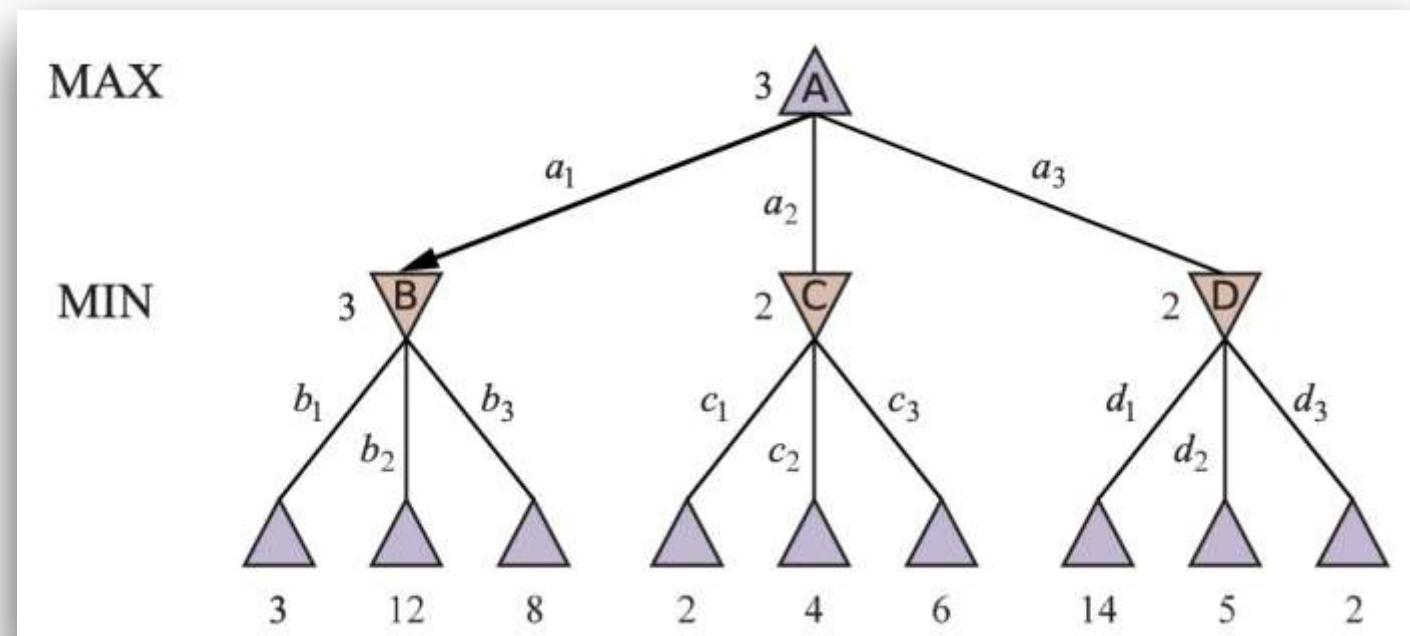
Algoritmo Minimax, considerazioni

Sfortunatamente, non possiamo eliminare l'esponente, ma possiamo almeno dimezzarlo: è di fatti possibile calcolare la decisione minimax corretta **senza** guardare tutti i nodi dell'albero di gioco.

Parleremo di **potatura**, ovvero la tecnica che consente di evitare di prendere in considerazione grandi porzioni dell'albero.

La specifica tecnica che esamineremo è chiamata **potatura alfa-beta**, che applicata ad un albero minimax restituisce lo stesso risultato della tecnica minimax standard, ma “pota” i rami che non possono influenzare la decisione finale.

Per capirne il funzionamento, consideriamo nuovamente l'albero di gioco discusso nella lezione precedente.



Potatura alfa-beta algoritmo minmax

L'algoritmo **Minimax** esplora tutto l'albero delle mosse per calcolare la decisione migliore, ma spesso visita rami che non servono. La **potatura alfa-beta** è un miglioramento che evita di esplorare quei rami inutili, mantenendo lo stesso risultato di Minimax ma più veloce.

L'idea di base

Mentre esplori l'albero:

α (**alfa**) = il miglior valore trovato finora per MAX lungo il cammino.

β (**beta**) = il miglior valore trovato finora per MIN lungo il cammino.

Se in un certo nodo ti accorgi che la valutazione corrente non potrà mai migliorare α o β , allora puoi **tagliare** quel ramo: non lo esplori perché sai già che il giocatore non lo sceglierà.

Come funziona con un esempio intuitivo

Immagina MAX deve scegliere tra due mosse:

Già nel primo ramo trova una possibilità di vittoria sicura (+1).

Quando passa a valutare il secondo ramo, MIN ha già un'alternativa che porta MAX a 0 o peggio.

A questo punto non ha senso esplorare il resto del secondo ramo: MIN non lascerà mai che MAX prenda di più, e MAX non sceglierà un cammino peggiore di +1.

Quindi quel pezzo dell'albero si **potatura**.

In parole povere

La potatura alfa-beta funziona come una forbice che taglia i rami "inutili":

Alfa tiene memoria del miglior risultato che MAX può già garantire.

Beta tiene memoria del miglior risultato che MIN può già garantire.

Se trovi un ramo che non può superare quei limiti, lo ignori.

Perché è utile

Stesso risultato di Minimax classico (stesse mosse scelte).

Ma esplora **molti meno nodi**.

Nel caso migliore, riduce la complessità da $O(b^d)$ a circa $O(b^{d/2})$ dove b è il branching factor e d la profondità).

In pratica: Minimax guarda ogni singolo ramo come un giudice pignolo, Alfa-Beta invece è un giudice furbo che, appena capisce che una mossa non potrà mai vincere il confronto, smette di considerarla.

Algoritmo Minimax con Potatura Alfa-Beta

La potatura alfa-beta può essere applicata ad alberi di qualunque profondità e spesso è possibile potare interi sotto-alberi, rendendo la potatura più efficace rispetto all'esempio fatto in precedenza.

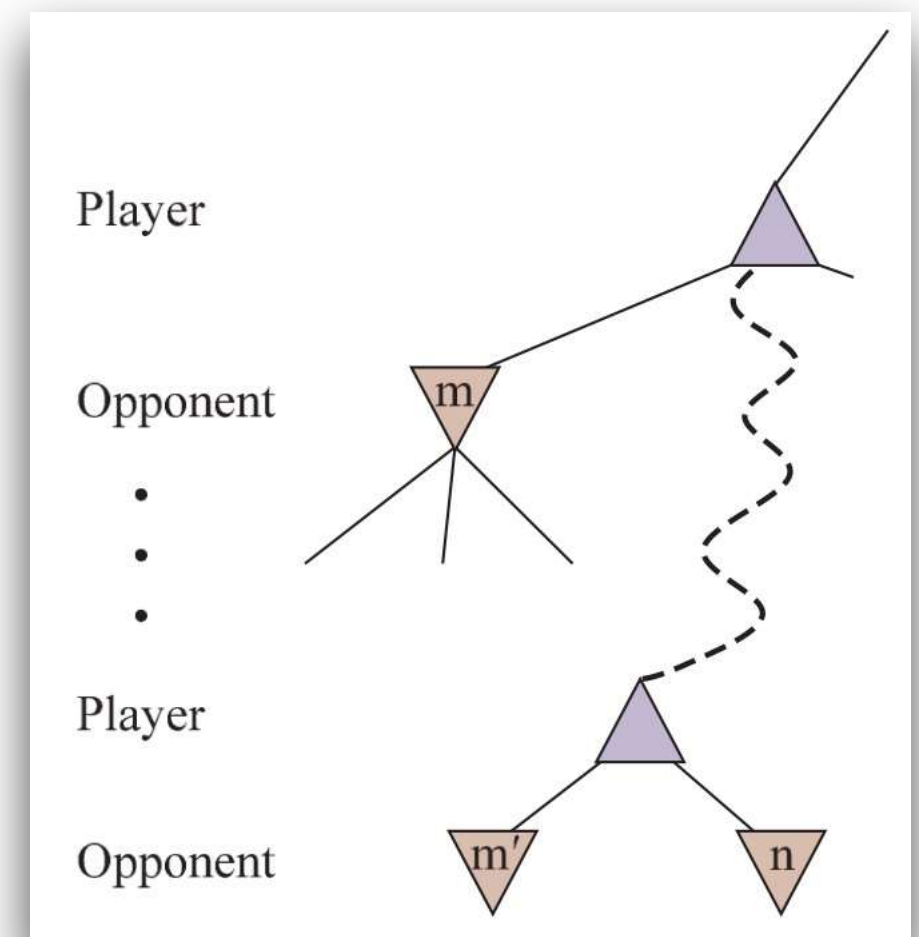
Il principio generale è questo: consideriamo un nodo n da qualche parte dell'albero tale che il giocatore abbia la possibilità di spostarsi in quel nodo.

Se esiste una scelta migliore m a livello del nodo padre o di qualunque nodo precedente, *allora n non sarà mai raggiunto in tutta la partita.*

Di conseguenza, possiamo potare n non appena abbiamo raccolto abbastanza informazioni per poter giungere a questa conclusione.

Alfa. Il valore della scelta migliore (quella con il valore più alto) per MAX che abbiamo trovato sin qui in un qualsiasi punto di scelta lungo il cammino.

Beta. Il valore della scelta migliore (quella con il valore più basso) per MIN che abbiamo trovato sin qui in un qualsiasi punto di scelta lungo il cammino.



Ricerca con Avversari

Algoritmo Minimax con Potatura Alfa-Beta

```
function RICERCA-ALFA-BETA(s) returns un'azione  
  v ← VALORE-MAX(s,  $-\infty$ ,  $+\infty$ )  
  return l'azione in AZIONI(s) con valore v
```

```
function VALORE-MAX(s) returns un valore di utilità  
  if TEST-TERMINAZIONE(s) then return UTILITÀ(s)  
  v ←  $-\infty$   
  
  for each a in AZIONI(s) do  
    v ← MAX(v, VALORE-MIN(RISULTATO(s, a), alfa, beta))  
    if v ≥ beta then return v  
    alfa ← MAX(alfa, v)  
  
  return v
```

```
function VALORE-MIN(s) returns un valore di utilità  
  if TEST-TERMINAZIONE(s) then return UTILITÀ(s)  
  v ←  $+\infty$   
  
  for each a in AZIONI(s) do  
    v ← MIN(v, VALORE-MAX(RISULTATO(s, a), alfa, beta))  
    if v ≤ alfa then return v  
    beta ← MIN(beta, v)  
  
  return v
```

Spiegazione pseudocodice minmax con potatura alfa-beta

Minimax quando aggiungi la **potatura alfa-beta**. Le differenze chiave sono: passi α e β lungo la ricorsione, li **aggiorni** dopo ogni figlio e **interrompi** l'esplorazione del nodo quando scatta la condizione di taglio.

Con la potatura alfa-beta i **cambiamenti principali** sono solo questi:

Ogni funzione ricorsiva riceve in input α e β .

VALORE-MAX-AB(s , α , β)

VALORE-MIN-AB(s , α , β)

Inizializzazione

In MAX: $v \leftarrow -\infty$

In MIN: $v \leftarrow +\infty$

Aggiornamento dei bound

In MAX: $\alpha \leftarrow \max(\alpha, v)$

In MIN: $\beta \leftarrow \min(\beta, v)$

Condizione di potatura (taglio)

In MAX: se $v \geq \beta \rightarrow$ ritorna subito v .

In MIN: se $v \leq \alpha \rightarrow$ ritorna subito v .

Cosa è cambiato rispetto al Minimax “puro”:

- Ogni funzione ricorsiva riceve α e β .

- Dopo ogni figlio si **aggiornano i bound** (α per MAX, β per MIN).

- Se $v \geq \beta$ in un nodo MAX (o $v \leq \alpha$ in un nodo MIN) si **ritorna subito**: i restanti figli sono inutili.

- La scelta dell'azione alla radice è identica; cambia solo quanta parte dell'albero esplori.

Suggerimento pratico: ordinare le mosse “migliori” per prime fa scattare i tagli prima e aumenta molto l'efficacia della potatura.

Algoritmo Minimax con Potatura Alfa-Beta, ordinamento delle mosse

L'efficacia della potatura alfa-beta dipende fortemente dall'ordinamento in cui gli stati sono esaminati.

Nel caso precedente, non abbiamo potuto potare alcun successore di D perché tutti i successori peggiori (per MIN) sono stati generati per prima.



Quindi, potrebbe essere una buona idea quella di esaminare per prima i successori *più promettenti* — ovvero, usando una strategia *best-first*. Se questo può essere fatto, la ricerca alfa-beta dovrà esaminare “solo” $O(b^{m/2})$ nodi - piuttosto che $O(b^m)$.

Una modifica di questo tipo consente alla ricerca alfa-beta di risolvere un albero profondo il doppio di quello che può risolvere minimax *nello stesso lasso di tempo*.

Talvolta, decidere un ordinamento è “semplice”. Negli scacchi, questa potrebbe essere una funziona tale che la ricerca abbia come obiettivo quello di catturare pezzi, poi di minacciarli, poi le mosse in avanti ed infine quelle all'indietro.

Algoritmo Minimax con Potatura Alfa-Beta, ordinamento dinamico delle mosse

Aggiungere schemi dinamici di ordinamento delle mosse, come provare per prima cosa le mosse che si sono rivelate le migliori in passato, ci porta vicino al limite teorico.

Un modo per ottenere informazioni dalla mossa corrente è quello di utilizzare una **ricerca ad approfondimento iterativo**.

Ricerca ad approfondimento iterativo: Con questa strategia, il limite viene incrementato progressivamente, finché un nodo obiettivo non è identificato.

Con questa strategia, si cerca innanzitutto uno strato in profondità e si registra il miglior cammino di mosse. Poi si cerca uno strato ancora in profondità, ma *si utilizza il cammino registrato allo scopo di fornire informazioni per l'ordinamento delle mosse*.

Le mosse migliori sono spesso chiamate *mosse killer* e si parla di *euristica della mossa killer* quando queste vengono provate per prime.

Adesso, però, la domanda vera è: *come utilizzare la ricerca ad approfondimento iterativo per l'ordinamento delle mosse?*

Ricordate che la ricerca ad approfondimento iterativo è **analoga a quella in ampiezza**, poiché ad ogni iterazione esplora completamente un livello di nodi prima di prendere in considerazione il successivo. Quindi, possiamo **esplorare i nodi in ampiezza e decidere come variare l'ordinamento in base ai valori minimax!**

Algoritmo Minimax con Potatura Alfa-Beta, ordinamento dinamico delle mosse

Un altro modo è quello di considerare le **trasposizioni**, che portano l'albero di gioco a contenere **cammini ridondanti** — stati ripetuti più volte nello spazio degli stati che possono determinare un aumento esponenziale del costo di ricerca.

Nella dama, ad esempio, il bianco può fare la mossa a_1 a cui il nero risponderà con b_1 . Allo stesso tempo, una mossa indipendente a_2 porterà alla mossa b_2 . Le sequenze $[a_1, b_1, a_2, b_2]$ e $[a_1, b_2, a_2, b_1]$ si concluderanno con la stessa configurazione!

Quindi, vale la pena *memorizzare la valutazione di una configurazione* in una tabella hash la prima volta che questa viene incontrata.

La tabella viene chiamata **tabella delle trasposizioni**: in essenza, questa è identica alla lista `esplorati` che abbiamo visto con gli algoritmi di ricerca non informata.

Proprio perché identica alla lista `esplorati`, possiamo decidere di ordinare le mosse successive sulla base della **conoscenza già acquisita esplorando una trasposizione**.

Dall'altra parte, se l'albero di gioco avesse milioni di stati, si arriverà necessariamente a dover mantenere in tabella solo alcune delle trasposizioni. Diverse strategie sono disponibili per selezionare le trasposizioni da mantenere in memoria.

Una strategia potrebbe essere derivata tramite la definizione di una funzione euristica.

Decisioni imperfette in tempo reale

L'algoritmo minimax genera l'intero spazio di ricerca, mentre quello alfa-beta ci permette di potarne buona parte. Tuttavia, anche alfa-beta deve condurre la ricerca fino agli stati terminali - almeno per una porzione dello spazio di ricerca.

Questa profondità **risulta normalmente non gestibile**, perché le mosse devono essere calcolate in un tempo ragionevole.

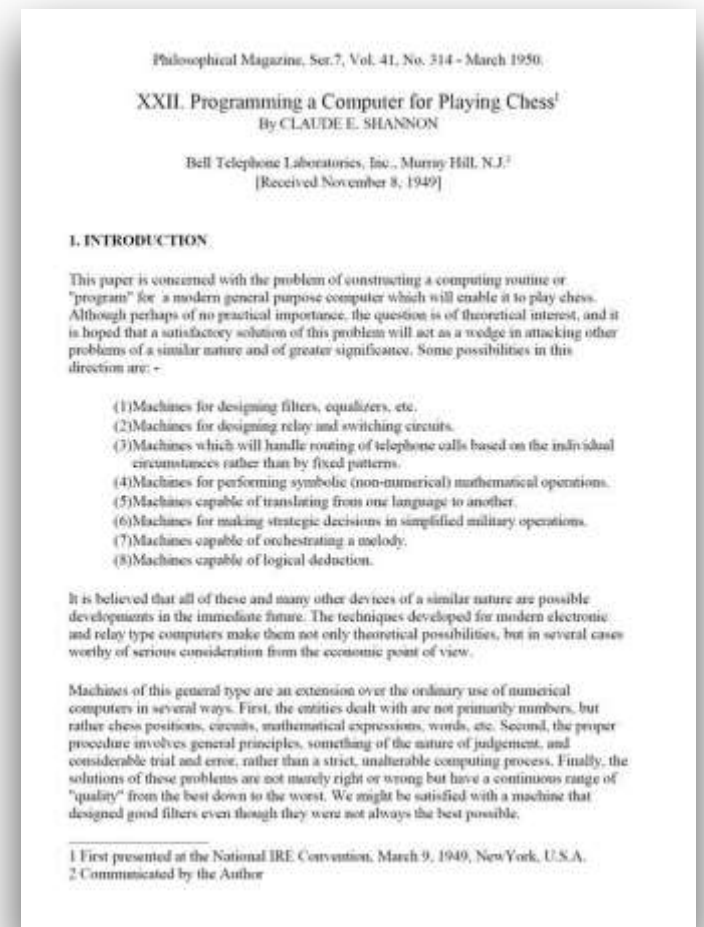
Claude Shannon (lo stesso Shannon della *formulazione di entropia*) nel 1950 propose che i programmi “tagliassero” la ricerca **prima di raggiungere le foglie applicando una funzione di valutazione euristica agli stati**.

In questa formulazione del problema, i nodi non terminali *diventano* foglie. Il minimax viene modificato in due punti:

La funzione di utilità viene sostituita da EVAL, che fornisce una stima dell'utilità della posizione raggiunta.

Il test di terminazione viene sostituito con un test di taglio (*cutoff test*), che decide quando applicare EVAL.

Ridefiniamo quindi il problema in questi termini.



Decisioni imperfette in tempo reale, ridefinizione del problema

La nuova formulazione porta il problema da:

$$\text{MINIMAX}(n) =$$

$$\left\{ \begin{array}{ll} \text{UTILITÀ}(s, \text{MAX}) & \text{se TEST-TERMINALE}(s); \\ \max_{a \in \text{AZIONI}(s)} \text{VALORE-MINIMAX-RISULTATO}(s, a) & \text{se GIOCATORE}(s) = \text{MAX}; \\ \min_{a \in \text{AZIONI}(s)} \text{VALORE-MINIMAX-RISULTATO}(s, a) & \text{se GIOCATORE}(s) = \text{MIN}; \end{array} \right.$$

alla seguente:

$$\text{H-MINIMAX}(n) =$$

$$\left\{ \begin{array}{ll} \text{EVAL}(s) & \text{se TEST-TAGLIO}(s, d); \\ \max_{a \in \text{AZIONI}(s)} \text{H-MINIMAX}(\text{RISULTATO}(s, a), d+1) & \text{se GIOCATORE}(s) = \text{MAX}; \\ \min_{a \in \text{AZIONI}(s)} \text{H-MINIMAX}(\text{RISULTATO}(s, a), d+1) & \text{se GIOCATORE}(s) = \text{MIN}; \end{array} \right.$$

Una funzione di valutazione restituisce una stima del guadagno atteso in una determinata posizione, **esattamente come le funzioni euristiche**.

Pertanto, la difficoltà di definizione di una funzione di valutazione è identica. D'altro canto, ogni giocatore di scacchi implementerà una propria strategia!

Decisioni imperfette in tempo reale, funzioni di valutazione

La funzione di valutazione dovrebbe in primo luogo *ordinare* gli stati terminali nello stesso modo della funzione di utilità. Gli stati che sono vittorie devono avere una valutazione migliore dei pareggi, che a loro volta devono essere migliori delle sconfitte.

La funzione di valutazione dovrebbe poi essere *veloce* da calcolare (altrimenti, non avrebbe senso sostituire la funzione di utilità!)

Terzo, per gli stati non terminali la funzione di valutazione dovrebbe avere una *forte correlazione con la probabilità reale di vincere la partita*.

La maggior parte delle funzioni di valutazione si basano sulle *caratteristiche* di uno stato: ad esempio, negli scacchi avremo come caratteristiche il numero di pedoni bianchi, il numero di pedoni neri, di regine bianche, di regine nere, e così via.

Le caratteristiche definiscono le cosiddette *classi di equivalenza*, che rappresenta un insieme di stati aventi lo stesso valore per tutte le caratteristiche. Ad esempio, tutti gli stati finali con due pedoni contro un pedone rappresentano una classe.

Ogni classe di equivalenza può contenere stati che portano alla vittoria, al pareggio e alla sconfitta. La funzione di valutazione non potrà sapere quali sono, ma potrà restituire un singolo valore che riflette la *proporzione di stati* che portano ad ogni risultato.

Portiamo avanti l'esempio degli scacchi.

Decisioni imperfette in tempo reale, funzioni di valutazione - l'esempio degli scacchi

Supponiamo che la nostra esperienza suggerisca che il 72% degli stati nella classe di equivalenza di due pedoni contro un pedone conducano ad una vittoria (utilità=+1), il 20% ad una sconfitta (0) e l'8% al pareggio (1/2).

In questo caso, una valutazione ragionevole per gli stati di questa classe di equivalenza potrebbe essere quella del *valore atteso*: $(0,72 \times 1) + (0,20 \times 0) + (0,8 \times 1/2) = 0,76$.

Si può, quasi sempre, determinare il valore atteso di ogni categoria, fornendo così una funzione di valutazione applicabile ad ogni stato.

Qualcuno potrebbe notare che una funzione di questo tipo richiede *un'eccessiva quantità di esperienza* in fase di definizione.

In molti casi, la funzione calcola valori separati per ogni caratteristica e poi li *combina* insieme per formare il valore finale. Nel caso degli scacchi, esiste il concetto di *valore del materiale*: ogni pedone vale 1, un cavallo o un alfiere 3, e così via. Altre caratteristiche come la difesa del re o una "buona" disposizione dei pedoni potrebbero avere altri valori, che però potrebbero essere semplicemente sommati.

Matematicamente, questo significa avere una funzione lineare pesata, poiché può essere espressa come:

$$\text{EVAL}(s) = w_1f_1(s) + w_2f_2(s) + \dots + w_nf_n(s); \text{ dove } w_i \text{ è il peso e } f_i \text{ è una caratteristica.}$$

Decisioni imperfette in tempo reale, terminare la ricerca

Come detto, la funzione di valutazione è una delle due modifiche necessarie al minimax. L'altra è quella relativa alla definizione del test di taglio.

Nella definizione del test di taglio, dobbiamo fare attenzione ai cosiddetti stati *non quiescenti* - stati che portano ad una variazione repentina del valore delle mosse.

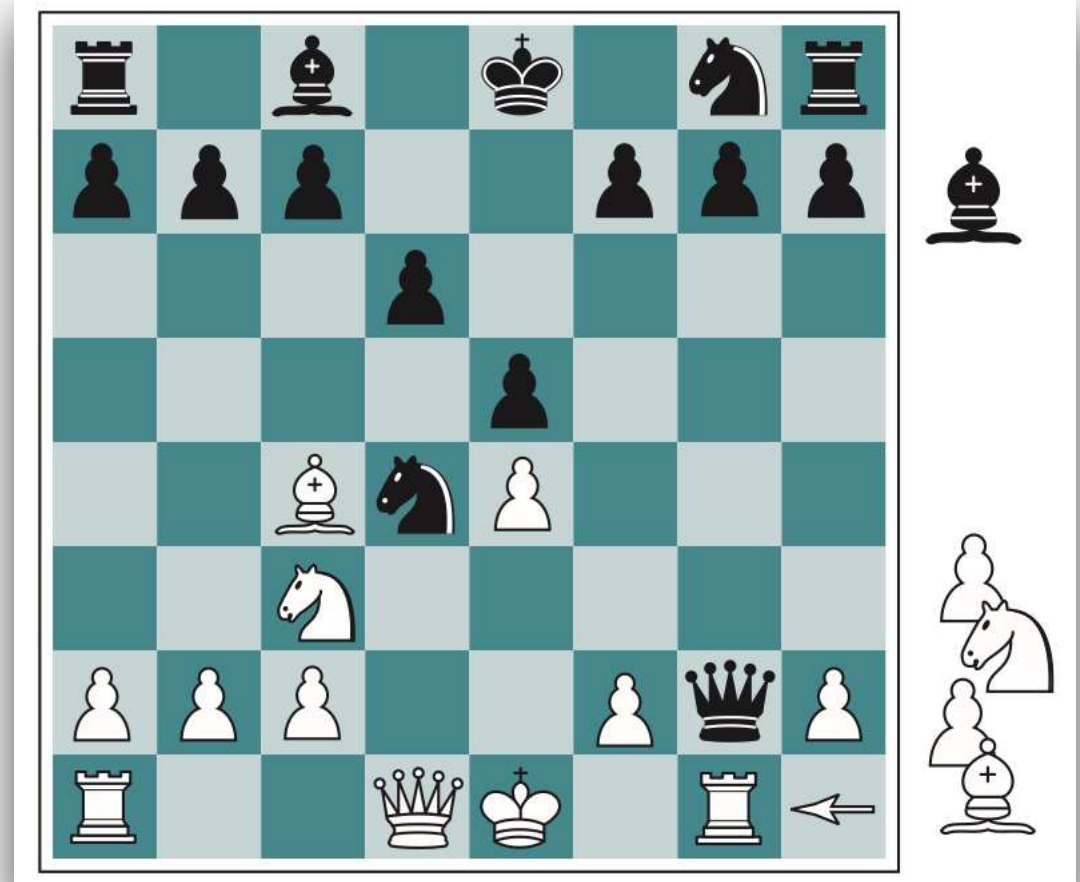
Supponiamo che, in una partita a scacchi, la funzione di valutazione sia basata sul vantaggio del materiale e che la ricerca sia arrivata al punto mostrato in figura.

Tocca al bianco muovere e il nero sembra essere in vantaggio, poiché ha un cavallo e due pedoni in più rispetto al bianco.

Il programma assocerebbe questo stato ad un valore euristico molto positivo, ritenendo che la situazione porterà ad una vittoria.

Ma la mossa successiva del bianco cattura la regina nera, portandolo in una situazione di vittoria quasi certa.

Questo è uno stato non quiescente, che porta ad un cambio netto dello stato di gioco.



Decisioni imperfette in tempo reale, terminare la ricerca

La funzione di valutazione dovrebbe perciò essere applicata solo a posizioni quiescenti, ovvero quelle per cui sia improbabile il verificarsi di grandi variazioni di valore nelle mosse immediatamente successive.

Talvolta, potrebbe essere inclusa nella definizione dell'algoritmo una *ricerca di quiescenza*, in cui stati non quiescenti vengono ulteriormente espansi fino a raggiungere posizioni quiescenti.

Da un punto di vista implementativo, le modifiche richieste all'algoritmo minimax con potatura alfa-beta sono minime.

```
function VALORE-MAX(s) returns un valore di utilità
  if TEST-TERMINAZIONE(s) then return UTILITÀ(s)
  if TEST-TAGLIO(s) then return EVAL(s)
  v ← -∞

  for each a in AZIONI(s) do
    v ← MAX(v, VALORE-MIN(RISULTATO(s, a), alfa, beta))
    if v ≥ beta then return v
    alfa ← MAX(alfa, v)

  return v
```


Minimax con potatura alfa-beta: ulteriori miglioramenti

Potatura in avanti. L'idea di base è quella di esplorare solo alcune mosse ritenute particolarmente promettenti, tagliando le altre.

Un approccio è la *beam search*: di volta in volta, si considerano solo le prime k migliori mosse, definite sulla base della funzione di valutazione. Tuttavia, non garantisce la non potatura delle mosse migliori:

I *tagli probabilistici* sono invece basati sull'esperienza: questo utilizza statistiche ricavate dalle precedenti esperienze per ridurre la possibilità che mossa migliore venga potata.

Database di mosse di apertura e chiusura. L'idea di base è che all'inizio di una partita ci sono solitamente poche mosse sensate e ben studiate, per cui è inutile esplorarle tutte. In maniera simile, per le fasi finali potrebbe essere possibile già predisporre delle chiusure così da non avere alcuna necessità di esplorare lo spazio.

I giochi di scacchi utilizzano tipicamente un database di questo tipo. All'inizio della partita, utilizzeranno mosse che sono “storicamente buone”, ovvero definite sulla base dell'osservazione dei più grandi scacchisti del mondo. Alla fine, avranno una maggior capacità di computazione rispetto all'uomo, essendo per questo capaci di definire delle strategie più efficaci/efficienti.

Dai giochi deterministici ai giochi stocastici

Nella vita reale si verificano molti eventi esterni che ci mettono in situazioni impreviste. Molti giochi modellano questa imprevedibilità includendo elementi casuali, come ad esempio l'uso di un dado. Si parla dei giochi stocastici.

Nel gioco del backgammon un giocatore deve lanciare due volte un dado per decidere quali mosse sono lecite.

In questo caso, chiaramente, l'algoritmo minimax o la ricerca alfa-beta *non sarebbero utilizzabili* nella loro forma base.

Possiamo però complementare gli algoritmi visti finora in maniera da introdurre dei **nodi possibilità** accanto a quelli di scelta. Così facendo, nel calcolare i valori di MIN e MAX dovremo tener conto delle probabilità dell'esperimento casuale.

In altri termini, non andremo più a calcolare i valori di massimo e minimo, ma i **valori di massimo e minimo attesi**. Da un punto di vista pratico, i nodi assumeranno un valore corrispondente alla somma del valore su tutti i risultati pesati in base alla probabilità di ciascuna azione.

I nodi terminali, quelli per cui è già noto il risultato dell'evento stocastico, saranno invece calcolati **nello stesso modo** visto precedentemente.

Dai giochi deterministici ai giochi stocastici

Un albero di gioco stocastico è rappresentato in figura. I nodi possibilità sono visualizzati come cerchi, quelli di scelta come triangoli.

Come detto, i nodi terminali sono calcolati secondo la regola standard dell'algoritmo minimax.

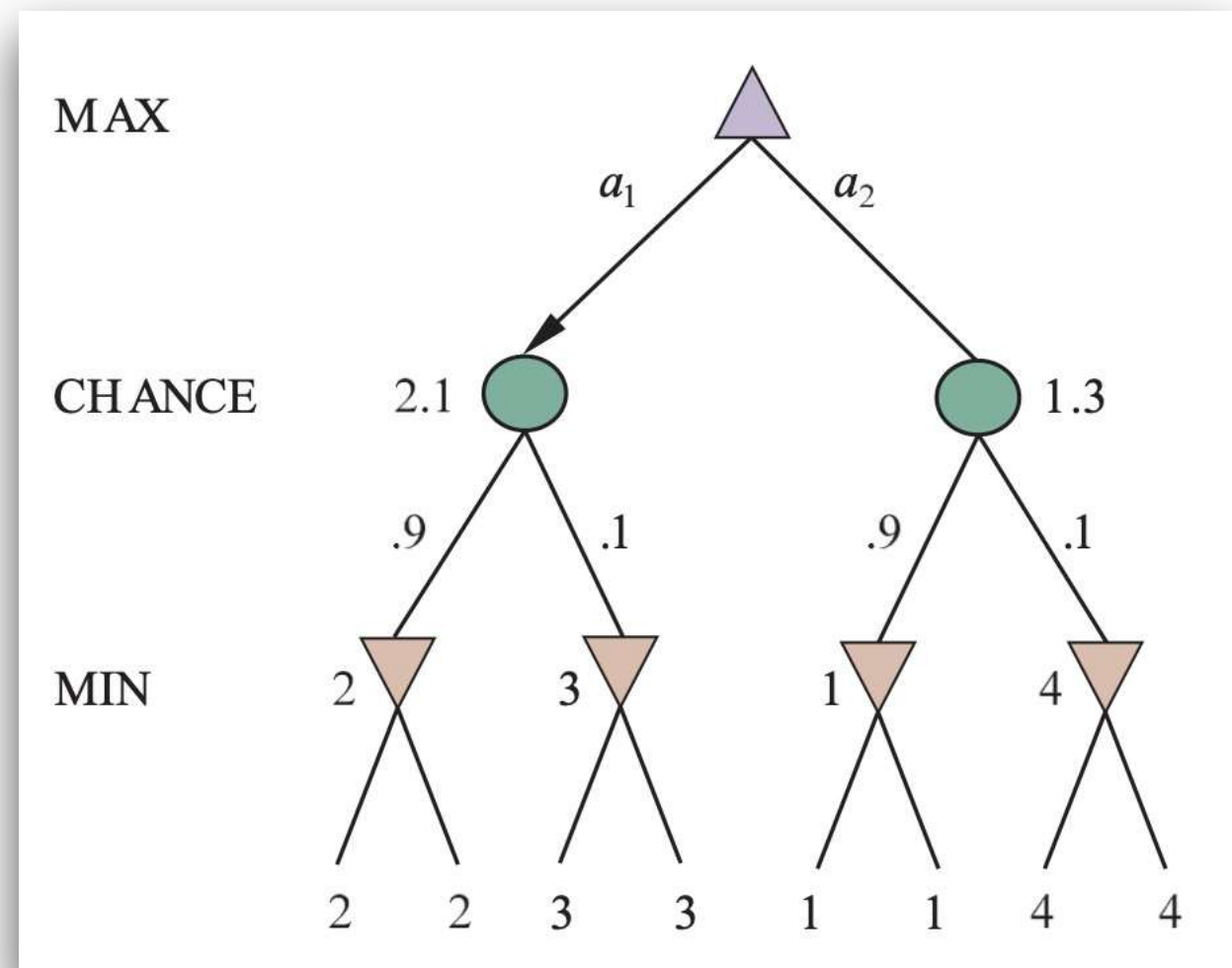
Dopodiché, sappiamo che MIN con probabilità 0.9 farà 2 e con probabilità 0.1 farà 3. Dall'altro lato, con probabilità 0.9 farà 1 e con probabilità 0.1 farà 4. Quindi, deriviamo i valori attesi:

$$0.9 \times 2 + 0.1 \times 3 = 2.1$$

$$0.9 \times 1 + 0.1 \times 4 = 1.3$$

Di conseguenza, la mossa migliore per MAX è a_1 .

Non avremo tempo di andare in dettaglio sulle caratteristiche di questi giochi, ma è fortemente consigliata la lettura del libro di testo, il quale propone una panoramica di come poter definire una funzione di valutazione per giochi con elementi casuali.



Dai giochi deterministici ai giochi parzialmente osservabili

In questa categoria di giochi, l'incertezza sullo stato di gioco nasce interamente all'**impossibilità di accedere alle scelte fatte dall'avversario**. Ad esempio, i giochi di carte in cui le carte iniziali dell'avversario non sono note.

Intuitivamente, potremmo pensare a questi giochi come un caso particolare dei giochi stocastici: potremmo avere un dado con tantissime facce all'inizio della partita ed esprimere i nodi possibilità per ogni possibile azione dell'avversario.

Sebbene l'analogia non è corretta, può servire per definire un algoritmo efficace. Consideriamo *tutte le possibili distribuzioni di carte nascoste* e risolviamo *una per una* come se fossero un gioco completamente osservabile. A quel punto, basterà scegliere la mossa che ha il *miglior risultato calcolato in media su tutte le distribuzioni*.

Questo equivale a calcolare:

$$\operatorname{argmax}_a \sum_s P(s) \operatorname{MINIMAX}(\operatorname{RESULT}(s, a))$$

Purtroppo però, in molti casi, il numero di possibili distribuzioni è troppo grande. Una soluzione consiste nell'applicazione di un'approssimazione Monte Carlo: invece di sommare tutte le distribuzioni, prendiamo un **campione casuale di N distribuzioni** in cui la probabilità di s di apparire nel campione è proporzionale alla probabilità $P(s)$.